

Project Proposal – AI-Powered Ransomware & Zero-Day Threat Mitigation Platform

Description:

AI-Powered Ransomware and Zero-Day Threat Mitigation Platform Using NLP Analytics, Hybrid Intrusion Detection, and Automated Disaster Recovery.

Team Members:

Dapo Omoworare	300379279
Oke Ebiyomare	300392124

Course: CSIS 4495 - 002 Applied Research Project

Team Lead: Sin-Yi Huang	300398274
-------------------------	-----------

1. Introduction	3
1.1 Background.....	3
1.2 Problem Statement.....	3
1.3 Literature Review.....	3
Security Log Analytics.....	3
Anomaly Detection.....	3
Ransomware Detection.....	3
Disaster Recovery.....	4
1.4 Research Gap.....	4
1.5 Research Hypotheses.....	4
1.6 Expected Benefits	4
2. Proposed Research Project	5
2.1 Research Objectives	5
2.2 Research Design and Methodology	5
Table 1: Four-layer architecture.....	5
NLP Log Processing	5
Machine Learning Detection.....	5
Cryptographic Integrity Verification	6
2.3 Technologies.....	6
Table 2: Technologies.....	6
2.4 Deliverables.....	8
Table 3: Deliverables	8
3. Project Planning and Timeline.....	8
Table 4: Project Schedule	8
4. Project Contract (Team Agreement)	8
5. AI Use Section	9
5.1 Table of AI Tools and Specific Use	9
Table 5. AI Tools and Specific Use	9
5.2 Value Addition.....	11
5.2 Appendix: Prompt History	11
6. Work Logs (Individual Contributions).....	11
Table 6. Teamwork logs	11
7. Closing and References	12
7.1 Acknowledgements.....	12
7.2 References	12

1. Introduction

1.1 Background

Cybersecurity threats continue to evolve in both frequency and sophistication, with ransomware and zero-day attacks representing two of the most significant risks facing modern organizations. Healthcare providers, financial institutions, and critical infrastructure operators increasingly rely on digital systems, making them attractive targets for cybercriminals. Traditional security tools such as Intrusion Detection Systems (IDS) and Security Information and Event Management (SIEM) platforms often focus on signature-based detection and struggle to identify previously unseen attack patterns.

At the same time, organizations generate enormous volumes of system logs from operating systems, applications, networks, and cloud services. Because these logs are frequently unstructured and contain significant noise, security analysts often experience alert fatigue, reducing the effectiveness of manual threat investigation efforts.

1.2 Problem Statement

This research seeks to address the following questions:

1. How can unstructured security logs from multiple sources be automatically processed and filtered to improve threat visibility?
2. How can a hybrid detection architecture combining rule-based intrusion detection and machine learning improve the identification of ransomware and zero-day threats?
3. How can disaster recovery mechanisms automatically restore services after a cyberattack with minimal downtime?
4. How can cryptographic integrity verification help identify file tampering and ransomware encryption activities?

These issues are important because delays in threat detection and recovery can result in operational disruptions, financial losses, and compromised organizational data.

1.3 Literature Review

Existing research has explored several related areas:

Security Log Analytics

Researchers have demonstrated the effectiveness of Natural Language Processing (NLP) techniques for transforming unstructured log data into structured security events. Methods such as tokenization, keyword extraction, and TF-IDF have been shown to improve event classification and reduce noise.

Anomaly Detection

Machine learning approaches such as Isolation Forests and Autoencoders have been widely applied to cybersecurity datasets to identify anomalous behaviors that may indicate previously unknown threats.

Ransomware Detection

Current ransomware detection studies focus on behavioral indicators including mass file encryption, unusual file access patterns, unauthorized privilege escalation, and lateral movement activities.

Disaster Recovery

Cloud-based recovery strategies increasingly leverage virtualization and containerization technologies to support automated failover capabilities and rapid restoration of compromised services.

1.4 Research Gap

Although significant research exists in log analytics, intrusion detection, anomaly detection, and disaster recovery, most solutions focus on individual components. Few integrated platforms combine:

- Multi-source log ingestion
- NLP-based data preprocessing
- Rule-based intrusion detection
- Machine learning anomaly detection
- Cryptographic integrity monitoring
- Automated recovery mechanisms

This research aims to address this gap through the development of a unified cybersecurity resilience platform.

1.5 Research Hypotheses

The following hypotheses will guide this research:

H1: NLP-based preprocessing can significantly reduce security event noise and improve analyst efficiency.

H2: A hybrid threat detection model combining signature-based detection and machine learning will achieve higher detection accuracy than either method independently.

H3: Automated failover and disaster recovery mechanisms can reduce system recovery times following ransomware incidents.

1.6 Expected Benefits

The proposed platform is expected to:

- Improve cybersecurity situational awareness.
- Reduce false-positive alerts.
- Enhance ransomware detection capabilities.
- Detect previously unknown attack behaviors.
- Accelerate disaster recovery processes.
- Improve organizational cyber resilience.

2. Proposed Research Project

2.1 Research Objectives

The primary objective of this project is to develop an AI-powered cybersecurity platform capable of detecting ransomware and zero-day attacks while automatically initiating recovery procedures. Specific objectives include:

1. Collect and process logs from multiple heterogeneous data sources.
2. Reduce irrelevant events using NLP preprocessing techniques.
3. Detect known attacks using signature-based security controls.
4. Identify previously unseen attacks using anomaly detection algorithms.
5. Verify critical file integrity through cryptographic hashing.
6. Trigger automated disaster recovery workflows.
7. Visualize security events through an executive dashboard.

2.2 Research Design and Methodology

The proposed system follows a four-layer architecture.

Table 1: Four-layer architecture

Layer	Description	Notes
Layer 1	Endpoint and Log Data Ingestion Layer	This layer aggregates security events from multiple sources: • Windows Event Logs • Linux Syslogs • PCAP Network Captures • CISA Known Exploited Vulnerabilities (KEV) Feeds • Software Bill of Materials (SBOM) Data The data is standardized and stored for analysis.
Layer 2	Zero-Trust Security and Intrusion Detection Layer	This layer performs initial threat assessment through: • STRIDE Threat Modeling • Snort Detection Rules • Suricata Detection Rules • Network Micro-Segmentation Known attack signatures and policy violations are identified at this stage.
Layer 3	AI Threat Analytics and Integrity Layer	This analytical layer consists of three components: NLP Log Processing Logs are transformed through: • Regular Expression Tokenization • TF-IDF Vectorization • Noise Filtering • Feature Extraction Machine Learning Detection Unsupervised machine learning models including Isolation Forest will evaluate behavioral anomalies and potential zero-day attacks.

Layer	Description	Notes
		Cryptographic Integrity Verification SHA-256 hashing functions will continuously verify system and backup integrity to detect: • Unauthorized modifications • File corruption • Ransomware encryption activity
Layer 4:	Cloud-Native Management and Dashboard Layer	This layer provides: • Security monitoring dashboards • Threat intelligence visualization • Recovery orchestration • Executive reporting

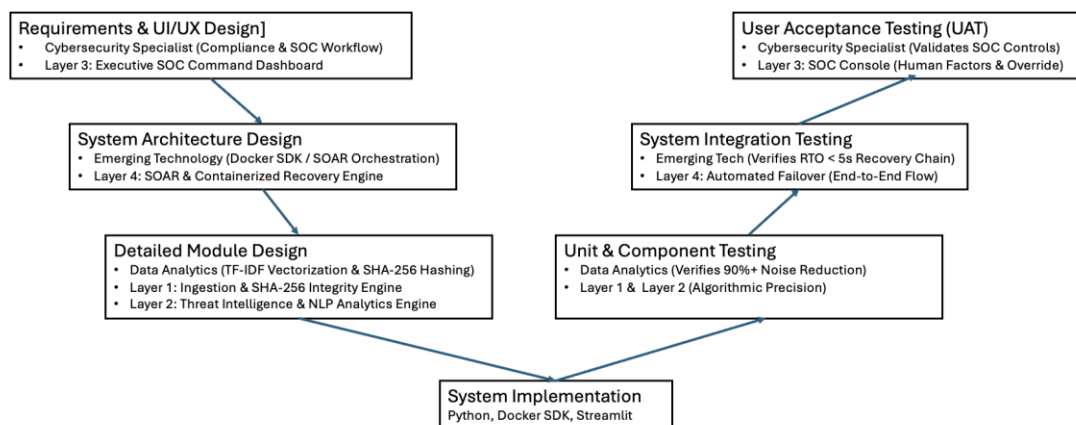
2.3 Technologies

Table 2: Technologies

Category	Technology	Note
Operating System	Ubuntu Linux	—
Platform	Docker, Docker Compose	—
Programming Language	Python	—
Framework	FastAPI	high-performance Python web framework used for building RESTful APIs quickly and efficiently
Database	PostgreSQL	An enterprise-grade relational database management system (RDBMS) that stores structured security events using standard SQL queries
Log Processing	PySpark	Python API for Apache Spark , designed for distributed, large-scale data processing and big data analytics (can be simulated with Pandas during local testing)
	Logstash	A data processing pipeline tool used to collect, parse, and ingest logs from multiple heterogeneous sources into a centralized database
NLP Tools	NLTK, spaCy	Natural Language Processing (NLP) libraries in Python designed for

		parsing, tokenizing, and analyzing unstructured textual log data
Machine Learning	Scikit-Learn	Python machine learning library used for data analysis, classification, and anomaly detection algorithms (e.g., Isolation Forest)
Intrusion Detection	Snort, Suricata	Open-source Network Intrusion Detection Systems (NIDS) that monitor network traffic using predefined signature-based rules to detect malicious activity
Dashboard	Power BI	A business intelligence platform by Microsoft used to visualize security metrics and create executive analytical dashboards
	Plotly Dash	A Python-based analytical web application framework used for building customized, interactive security dashboards
Cryptography	SHA-256	Cryptographic hash function that generates a unique 256-bit signature for files to detect unauthorized tampering or ransomware encryption. (Executed natively in Python via hashlib)
Version Control	GitHub	—

Figure 1 V model



2.4 Deliverables

The project will produce the following deliverables:

Table 3: Deliverables

Deliverable	Context
Deliverable 1	Multi-Source Log Ingestion Pipeline
Deliverable 2	NLP-Based Log Processing Module
Deliverable 3	STRIDE Threat Modeling Framework
Deliverable 4	Hybrid Intrusion Detection Engine
Deliverable 5	Machine Learning Zero-Day Detection Engine
Deliverable 6	SHA-256 Integrity Verification Service
Deliverable 7	Executive SOC Analytics Dashboard
Deliverable 8	Final Research Report and Technical Documentation

3. Project Planning and Timeline

3.1 Project Schedule

Table 4: Project Schedule

Week	Activity	Deliverable
1	Architecture Design and Environment Setup	Development Environment
2	Database Design and STRIDE Modeling	Threat Model
3	Proposal Submission	Project Proposal
4	NLP Parsing Development	NLP Engine
5	ML Detection Development	Progress Report 1
6	Disaster Recovery Development	Recovery Module
7	System Integration	Midterm Report
8	Security Stress Testing	Test Results
9	Dashboard Development	Progress Report 2
10	Recovery Optimization	Progress Report 3
11	Final Testing and Documentation	Progress Report 4
12	Final Demonstration and Submission	Final Project

4. Project Contract (Team Agreement)

- **Team Roles**

Our team will follow a simple Agile (Scrum-style) workflow. One member will act as the Team Lead, who is responsible for coordinating submissions, organizing meetings, and keeping the team on track. All team members will share responsibility for research, design, and documentation.

- **Communication**

We will use Discord as our main communication tool for daily updates and quick discussions. Email will be used for formal communication if needed.

Project files will be shared and updated in Google Drive and GitHub so everyone can easily access the latest version.

- **Meetings & Workflow**

- The team will work in short cycles.
- We will have one weekly meeting to review progress, discuss problems, and plan the next tasks.
- Short check-ins may happen if urgent issues come up.
- Meetings will be short and focused on what needs to be done next.

- **Task Division**

- Tasks will be divided during sprint planning. Each team member is responsible for:
 - Updating their individual work log regularly.
 - Inform the team early if they face any problems.
 - Team members can help each other if needed, but each task has a clear owner.

- **Conflict Resolution**

If there is a disagreement, the team will first discuss it openly and respectfully. If no agreement is reached, the Team Lead will help make a final decision based on project goals, scope, and course requirements.

- **Participation & Deadlines**

- All team members are expected to:
 - Attend meetings
 - Participate actively
 - Meet agreed deadlines

If someone cannot complete a task on time, they must inform the team as early as possible so adjustments can be made.

- **Agreement**

By signing below, all team members agree to follow this team contract and work together responsibly throughout the project.

Team Member Signatures:

Omowore	<i>OLADAPO</i>
Oke	<i>OKE</i>
Sin-Yi Huang	<i>Sin-Yi Huang</i>

5. AI Use Section

5.1 Table of AI Tools and Specific Use

Table 5. AI Tools and Specific Use

AI Tool Name	Version & Account Type	Specific Use
ChatGPT	GPT-4o	1. System Architecture & Data Flow Design: Evaluates the logical flow across the four-layer platform architecture (Layer 1 Ingestion through Layer 4 Containerized Recovery) and assists in mapping log parsing pipelines

		2.Rapid Code Generation: Produces Minimal Viable Product (MVP) Python boilerplate code for FastAPI routes, Scikit-Learn pipelines (IsolationForest, TfidfVectorizer), and Docker Compose configurations.
Claude	Claude 3.5 Sonnet	1.Academic & Technical Writing: Leverages long-context reasoning to synthesize dense technical literature, refine the project proposal, and structure research hypotheses. 2.Deep Code Review & Refactoring: Audits core Python modules and data processing scripts to ensure modularity, PEP 8 compliance, and robust error handling
Copilot	OpenAI GPT-4 / GPT-4o	VS Code Inline Completion: Integrates directly into the development environment (VS Code) to provide real-time code auto-completion while writing Python scripts, SQL queries, and Dockerfiles
Grammarly	Free Version	Advanced Proofreading: Identifies and corrects tense inconsistencies, dangling modifiers, and stylistic errors across all formal documentation
Google Gemini	Gemini (account type TBD) Gemini 1.5 Flash	Used during early project scoping to brainstorm candidate project concepts (IoMT vs. Ransomware platforms), identify real-world motivating incidents (Change Healthcare/BlackCat), draft English-language system architecture diagrams and the four-layer methodology, compare project novelty and workload, and draft an initial 10-week parallel timeline before this proposal's final 12-week schedule was set.

5.2 Value Addition

AI tools were used for brainstorming and language support.
The team added value by:

- Defining the core concept and scope
- Making design decisions
- Structuring the emotional reflection framework
- Ensuring spoiler-sensitive interaction design

5.2 Appendix: Prompt History

The following is a summary of the prompts used with Google Gemini during project brainstorming and early proposal scoping. Full chat logs are retained by the team and available on request.

- Asked for the names of major news reports covering the 2024–2025 healthcare ransomware attacks (BlackCat/ALPHV) targeting diagnostic data channels — used to identify real-world background sources and an APA citation format for the proposal's motivating case study.
- Asked for two project ideas suited to a team with Emerging Technology, Data Analytics, and Cybersecurity streams, building on the Change Healthcare/BlackCat case — used to generate the initial concepts for the IoMT anomaly-detection project and this Ransomware & Zero-Day Threat Mitigation project.
- Asked whether the IoMT concept could be scaled down into a mobile/web app — used to explore a product-focused MVP framing and app-based demo concept for the alternative project.
- Asked for an English-language system architecture for the IoMT project, and raised the constraint that the team has no access to real hospital data — used to identify Synthea, CICIDS2017, and the ECU IoMT dataset as compliant public/synthetic data sources.
- Asked for an English-language system architecture for this project (AI-Powered Ransomware & Zero-Day Threat Mitigation Platform) — used to draft the four-layer architecture, stream responsibilities, and methodology narrative that this proposal builds on.
- Asked which of the two candidate projects represented more work — used to inform team-size allocation and scoping discussions.
- Asked for a novelty comparison between the two candidate projects, then for that comparison translated into English — used to compare the projects during selection, prior to the team finalizing this project.
- Asked whether the two candidate projects could run independently and requested individual 10-week timelines for each — used to draft an initial two-track, 10-week schedule during scoping, before this proposal's final 12-week single-project timeline was adopted.

6. Work Logs (Individual Contributions)

A separate work log must be included for each team member

Table 6. Teamwork logs

Team member	Date	Number of Hours	Description of Work Done	Contribution
Dopa				

Team member	Date	Number of Hours	Description of Work Done	Contribution
Oke				
Sin-Yi				

7. Closing and References

7.1 Acknowledgements

The team would like to thank the course instructor for their guidance and feedback throughout the project planning process.

7.2 References

Ahmad, S., & Alsmadi, I. (2021). Log analysis and anomaly detection using natural language processing. *Journal of Information Security and Applications*, 61, Article 102919. <https://doi.org/10.1016/j.jisa.2021.102919>

Barker, E. (2020). *Recommendation for key management: Part 1 – General* (NIST Special Publication 800-57 Part 1 Rev. 5). National Institute of Standards and Technology. <https://doi.org/10.6028/NIST.SP.800-57pt1r5>

Docker Inc. (2024). *Docker SDK for Python documentation* (Version 7.0.0) [Software documentation]. <https://docker-py.readthedocs.io/>

National Institute of Standards and Technology. (2024). *The NIST Cybersecurity Framework (CSF) 2.0* (NIST Special Publication 1299). U.S. Department of Commerce. <https://doi.org/10.6028/NIST.SP.1299>

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12(85), 2825–2830. <http://jmlr.org/papers/v12/pedregosa11a.html>

Ramos, J. (2003). Using TF-IDF to determine word relevance in document queries. In *Proceedings of the First Instructional Conference on Machine Learning* (Vol. 242, pp. 29–48). Integrated Knowledge Properties.

Rose, S., Borchert, O., Mitchell, S., & Connelly, S. (2020). *Zero Trust architecture* (NIST Special Publication 800-207). National Institute of Standards and Technology. <https://doi.org/10.6028/NIST.SP.800-207>

Strom, B. E., Applebaum, A. L., Miller, B. D., Nickels, K. C., Pennington, A. G., & Thomas, C. B. (2018). *MITRE ATT&CK: Design and philosophy* (Technical Report MTR180189). The MITRE Corporation. <https://www.mitre.org/sites/default/files/publications/pr-18-0944-11-mitre-attack-design-and-philosophy.pdf>

Tulu, M. M., Zander, S., & Bhamare, D. (2023). Security orchestration, automation, and response (SOAR): A survey of techniques, tools, and challenges. *IEEE Access*, 11, 45210–45231. <https://doi.org/10.1109/ACCESS.2023.3273100>

Zheng, L., & Li, Y. (2022). Anomaly detection and container failure recovery in cloud-native microservices architecture. *IEEE Transactions on Network and Service Management*, 19(3), 3102–3115. <https://doi.org/10.1109/TNSM.2022.3178201>